PV-Finder: primary vertex reconstruction and track association with neural networks

ANA-IDTR-2026-01, Editorial Board

Qi-Bin Lei, Rocky B. Garg, Lauren Tompkins,
Mattia Mauro, Trey Warner

Stanford University

November 13, 2026







Motivation

Find the primary vertices and their tracks at the HL-LHC.

- About 200 interactions per crossing along a few cm of beam line.
- PV-Finder: a neural network turns tracks into a vertex-density histogram.
- A graph neural network (GNN) assigns tracks to the vertices.
- Compared with AMVF on the same events, with the same definitions.

$$\langle \mu \rangle = 200$$

HL-LHC, ITk

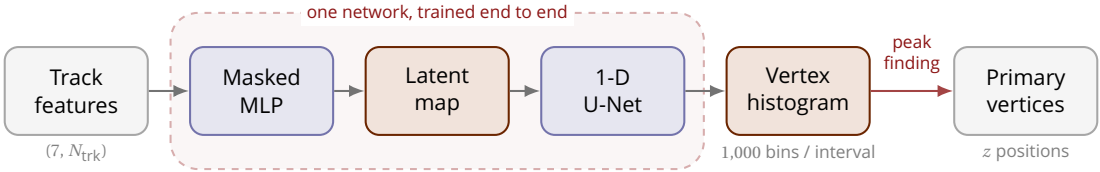
I. Method
II. Vertex finding at PU200
III. Track association and hard scatter
IV. Low pile-up and collision data
V. Timing and deployment

I

Method

Tracks, network, peaks and track association

From Tracks to a Vertex Histogram



Inputs

$d_0, z_0, \sigma_{d_0}, \sigma_{z_0}, \mathrm{cov}(d_0, z_0)$ and the interval bounds.

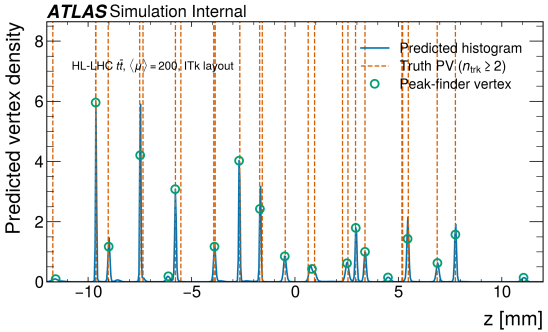
Network

Per-track MLP, summed over tracks, then a U-Net. Independent of track order.

Output

Twelve 40 mm intervals: $12{,}000$ bins of 0.04 mm over $|z| < 240$ mm.

Peak Finding and the Working Point



One configuration for all HL-LHC results

Amplitude threshold	0.01
Minimum width	3 bins
Minimum integral	0.30
Minimum height	0.03
Position	local centroid
GNN score cut t	0.98
Matching window	0.17 mm
Hard-scatter window	0.1 mm

Peaks split at local minima; centroid over ± 3 bins of the maximum.

Track-to-Vertex Association with a GNN

Track nodes: d_0, z_0, errors, θ, ϕ, p_T.

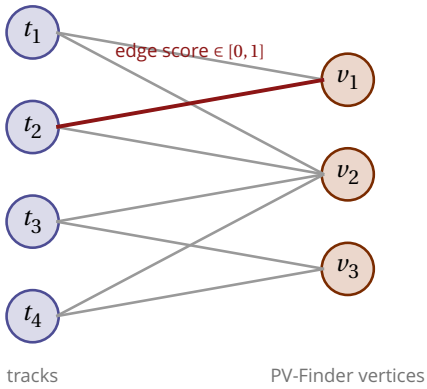
Vertex nodes: PV-Finder position and peak height.

Edges: each track to its $k = 20$ nearest vertices, with

$$\frac{z_0 - z_{\mathrm{PV}}}{\sqrt{\sigma_{z_0}^2 + \sigma_{\mathrm{PV}}^2}}, \quad \frac{d_0}{\sigma_{d_0}}, \quad |z_0 - z_{\mathrm{PV}}|.$$

Output: a score per edge. A track goes to its best vertex if the score passes $t = 0.98$.

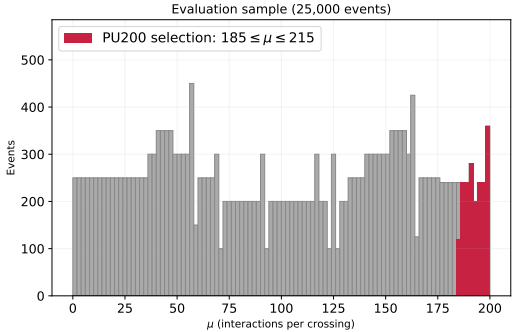
Graph attention network, $39{,}521$ parameters.



HL-LHC Samples

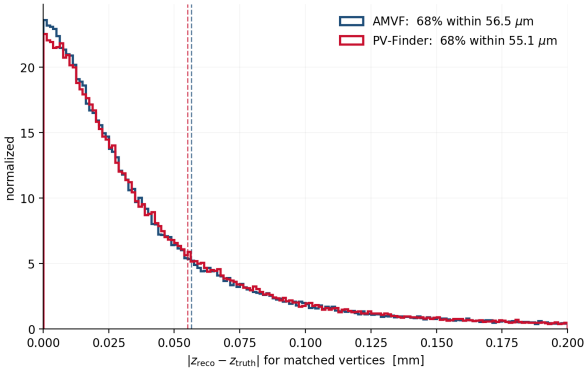
$\sqrt{s} = 14$ TeV $t\bar{t}$, ITk, $|\eta| < 4$

- Density network: 5.17M training events near $\mu = 200$.
- GNN: 180,000 training events.
- Evaluation on held-out files, flat in μ from 0 to 200.
- PU200 results: 1,920 events at $185 \leq \mu \leq 200$, $\langle\mu\rangle = 192.5$.
- 111.3 truth vertices per event with ≥ 2 tracks.



Tracks: $p_T \geq 0.9$ GeV, $|d_0| \leq 1$ mm, $\sigma(d_0) \leq 0.35$ mm, $|z_0| \leq 200$ mm.

Matching Window: 0.17 mm



Single-vertex position residual

68% of matched vertices lie within:

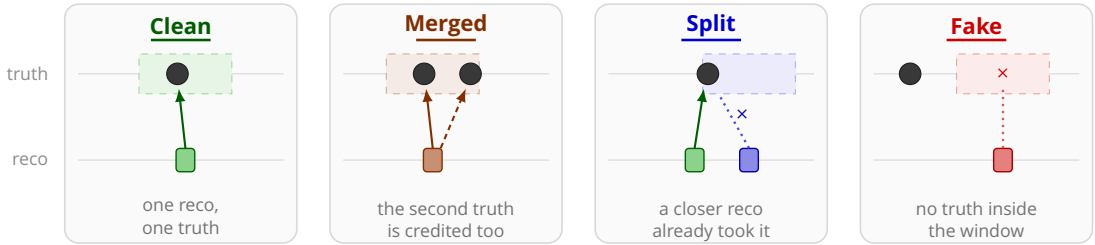
PV-Finder	$55.1\ \mu$m
AMVF	$56.5\ \mu$m

Window $= 3 \times 56.5\ \mu\text{m} \simeq 0.17$ mm, the same for both algorithms.

Hard scatter: 0.1 mm, as in the ITk performance studies.

Vertex Categories

Match reconstructed to truth vertices by z, closest first, one to one.



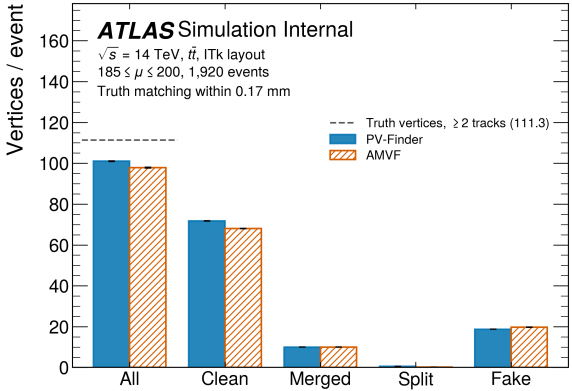
Efficiency: truth vertices with a reconstructed vertex within the window (clean or merged) / all truth vertices with ≥ 2 tracks.

II

Vertex finding at PU200

Before track association, same events as AMVF

PU200: PV-Finder against AMVF



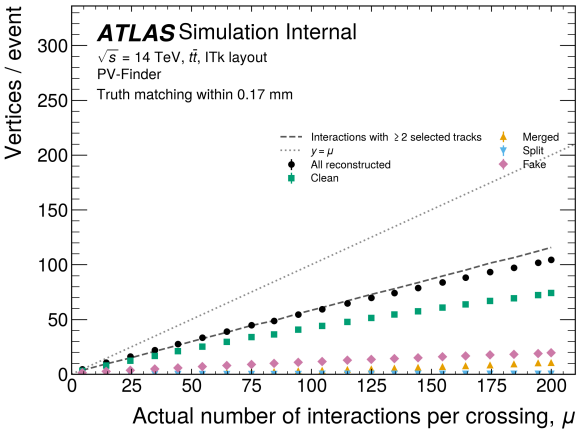
	PV-Finder	AMVF
Efficiency [%]	83.2	79.8
Fake / event	18.7	19.7
Split / event	0.53	0.18

$$+3.36 \pm 0.06$$

points of efficiency over AMVF

Hard-scatter window (0.1 mm): 74.1% vs 71.0%.

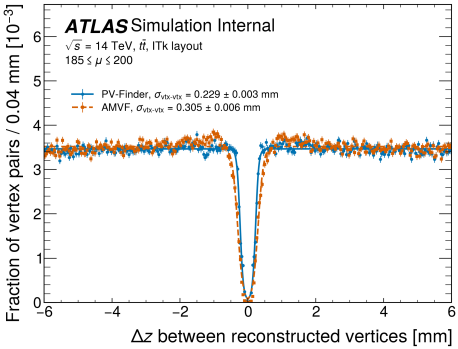
Vertex Yield across Pile-up



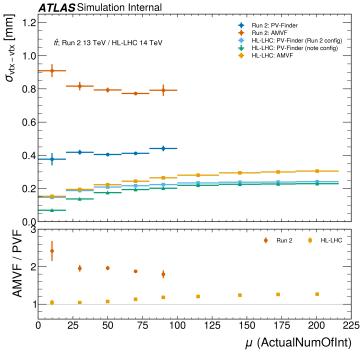
Reconstructed vertices per event, by category

Clean vertices dominate at every μ up to 200.

Vertex–Vertex Resolution



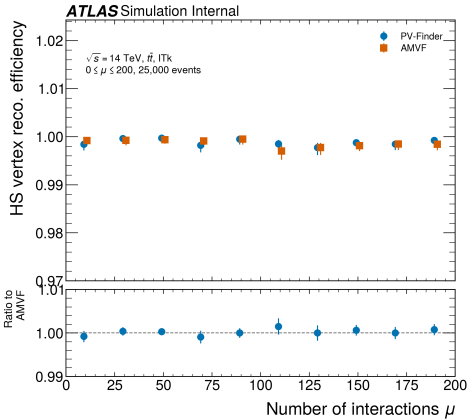
Δz between reconstructed vertices, PU200



$\sigma_{\mathrm{vtx-vtx}}$ versus μ

PU200: $\sigma_{\mathrm{vtx-vtx}} = 0.218 \pm 0.003$ mm for PV-Finder, 0.305 ± 0.006 mm for AMVF.

Hard-Scatter Vertex Reconstruction



A reconstructed vertex within 0.1 mm of the hard-scatter vertex.

99.9%

PV-Finder at PU200

99.8%

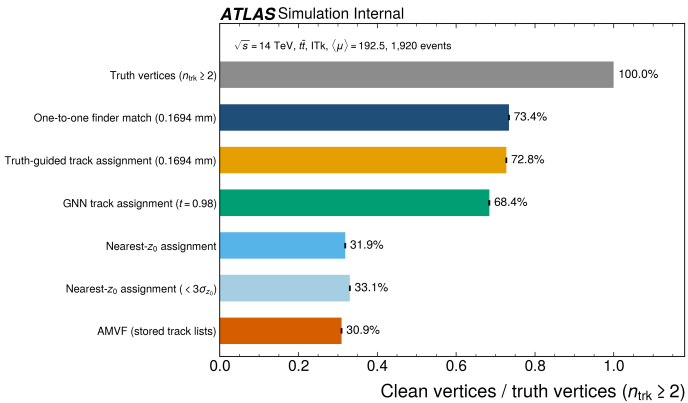
AMVF

III

Track association and hard scatter

After the GNN: categories, hard scatter and position

Full Chain at PU200



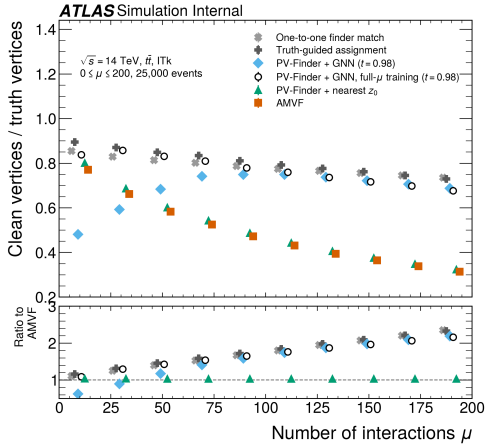
Clean vertices / truth vertices

Clean: $\geq 70\%$ of the vertex tracks from one interaction.

PV-Finder + GNN	68.4%
Nearest z_0	31.9%
z_0 within 3σ	33.1%
AMVF	30.9%

The GNN more than doubles the clean-vertex yield of AMVF and of z_0-based assignment.

Full Chain across Pile-up



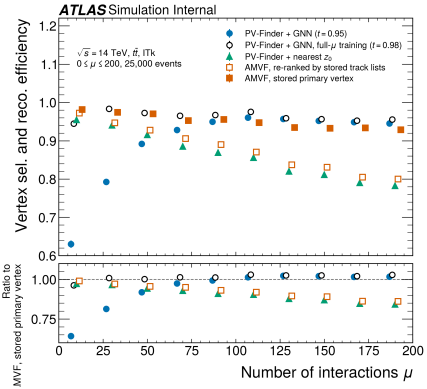
Clean vertices / truth vertices versus μ for each stage of the chain and for AMVF.

Lower panel: ratio to AMVF.

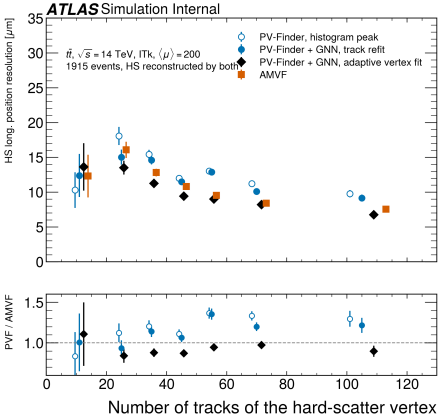
Hard-Scatter Vertex after Track Association

PU200	GNN	AMVF
Clean HS vertex	96.4%	92.5%
HS tracks on HS vertex	74.9%	74.1%
HS vertex purity	91.3%	88.4%
HS selected ($\sum p_T^2$)	96.3%	92.9%

ATLAS definitions; tracks counted, not fit-weighted. HS vertex: most HS tracks. Selected: largest $\sum p_T^2$ and within 0.1 mm. AMVF: its stored primary vertex.



Hard-Scatter Longitudinal Resolution



Vertex fit on the GNN tracks

Adaptive 3D fit with the beam-spot constraint, seeded at the PV-Finder peak.

ITk definition: iterative RMS of $z_{\text{reco}} - z_{\text{HS}}$, $1{,}915$ events with the HS found by both.

$$\mathbf{9.7 \pm 0.3\ \mu m}$$

PV-Finder + GNN

$$10.2 \pm 0.3\ \mu\text{m}$$

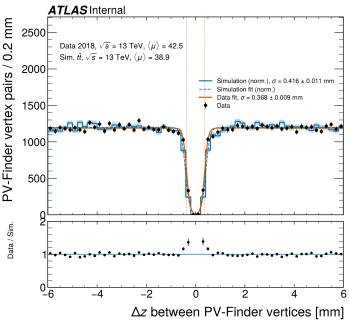
AMVF

IV

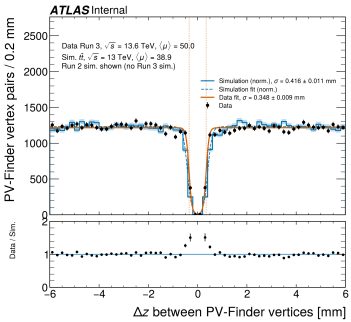
Low pile-up and collision data

Run 2 simulation, Run 2 and Run 3 data

Low Pile-up: Simulation and Collision Data



Run 2 data and simulation



Run 3 data and simulation

	Simulation	Run 2 data	Run 3 data
PV-Finder $\sigma_{\text{vtx}-\text{vtx}}$ [mm]	0.416	0.368	0.348

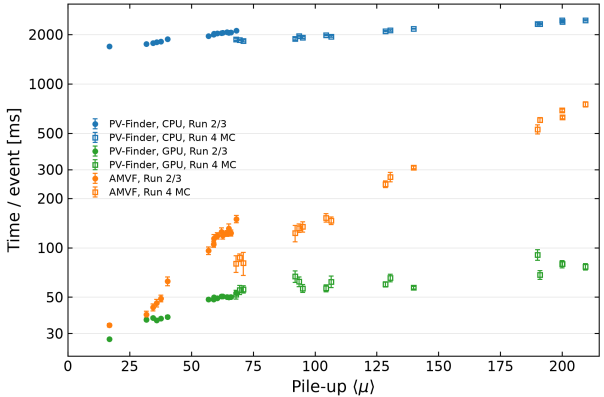
Simulation: efficiency 91.6%, 0.42 fake vertices per event; AMVF $\sigma_{\text{vtx}-\text{vtx}} = 0.789$ mm.

V

Timing and deployment

Athena integration, CPU and GPU

Athena: Timing on CPU and GPU



Same events, tracks to vertices with track links

Intel Xeon Silver 4216, NVIDIA T4.

At $\mu = 200$	ms/event
PV-Finder + GNN, GPU	76
PV-Finder + GNN, CPU	2401
AMVF, CPU	~ 650

GPU chain $\sim 9\times$ faster than AMVF.

Implemented in Athena by Trey Warner. Tools and validation in the backup.

A Smaller Network for CPU

Same data and loss, narrower U-Net

	v6	s5	s8
U-Net channels	280	112	80
CPU time [ms]	1,601	250	178
Speed-up	1	6.4×	9.0×
Δefficiency [pt]		−0.33	−0.80
Δfake / event		−0.9	−3.1

ONNX Runtime, one thread, one PU200 event. Small models use packed inference: tracks only, no padding, identical output.

Projected Athena time at $\mu = 200$, CPU

Chain	ms/event
v6 + GNN	2401
s5 + GNN	∼ 555
s8 + GNN	∼ 455
AMVF	∼ 650

Packed inference alone: 1.4× for v6 at no cost.

The small network brings the CPU chain to the speed of AMVF at $\mu = 200$.

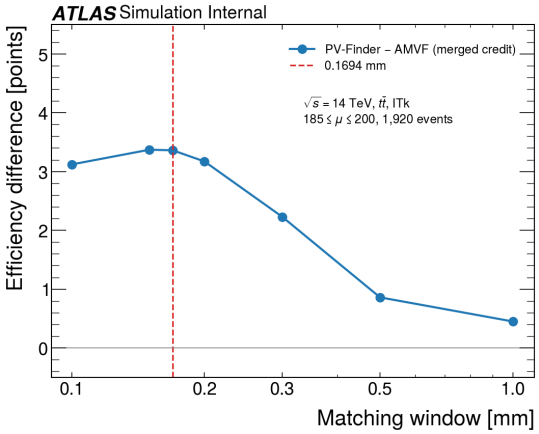
Summary

- At PU200, PV-Finder finds 83.2% of truth vertices, $+3.36$ points over AMVF, with fewer fakes.
- Vertex–vertex resolution 0.218 mm against 0.305 mm for AMVF.
- With GNN track association: 68.4% clean vertices per truth vertex, against 30.9% for AMVF.
- Hard scatter: selected in 96.3% of events (92.9% AMVF); position resolution 9.7 μm (10.2 μm AMVF).
- Validated on Run 2 and Run 3 collision data.
- Running in Athena on CPU and GPU.

Next: freeze the HL-LHC density model and GNN, and complete the paper draft.

Backup

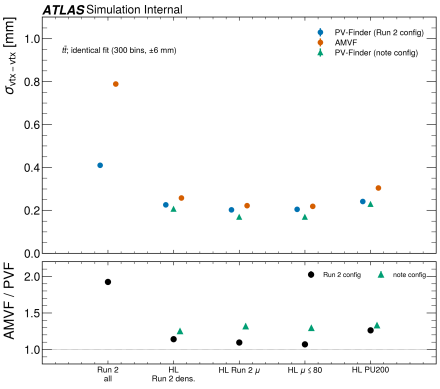
Backup: Efficiency Difference versus Matching Window



PV-Finder minus AMVF efficiency, same events, for windows from 0.1 to 1 mm.

PV-Finder is ahead at every window.

Backup: Run 2 and HL-LHC Resolution



AMVF / PV-Finder $\sigma_{\text{vtx}-\text{vtx}}$: 1.92 in Run 2 simulation, 1.40 at PU200.

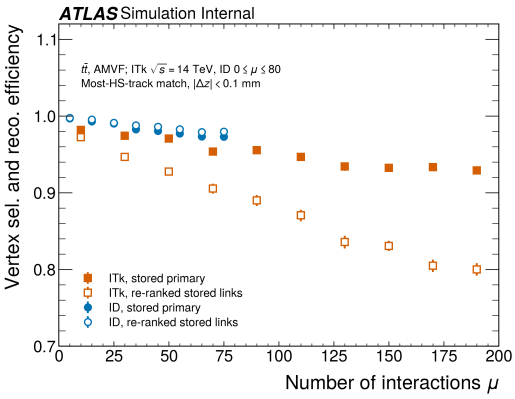
Track-based vertex precision: 75.6 μm (Run 2), 44.4 μm (HL-LHC).

Training target through the same peak finding:

$\sigma_{\text{vtx}-\text{vtx}}$ [mm]	Run 2	PU200
Target	0.203	0.145
PV-Finder	0.410	0.218
PV-Finder / target	2.0	1.5

A 0.02 mm grid moves the target from 0.137 to 0.124 mm.

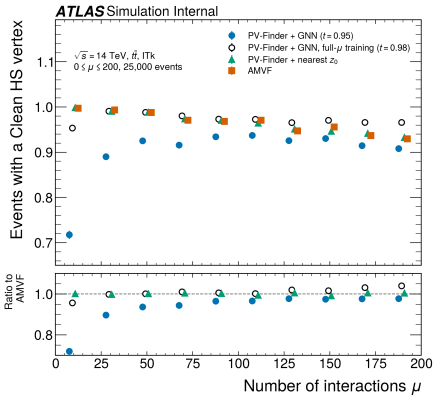
Backup: AMVF Hard-Scatter Selection

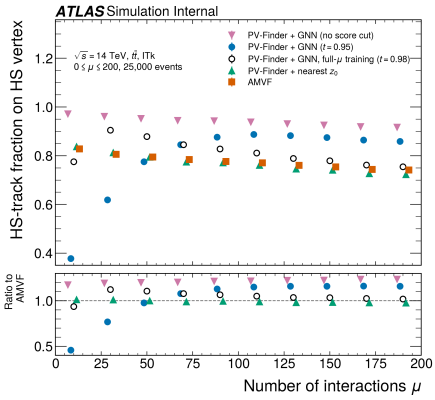


AMVF stored primary vertex: 92.9% at PU200, as in the ITk performance paper.

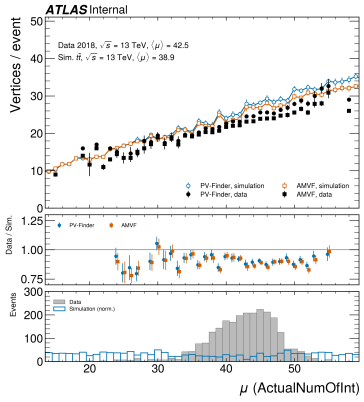
Re-ranking the stored track lists by $\sum p_T^2$ gives 80.7%: a few very high-p_T tracks change the leading vertex.

Backup: Hard-Scatter Tracks and Clean HS versus Pile-up





Backup: Low Pile-up Vertex Yield and the Reference

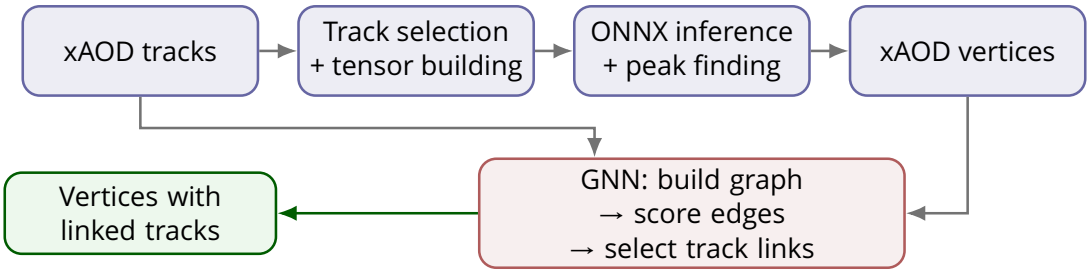


PV-Finder peaks without a reference vertex, per event

Simulation, truth reference	0.42
Simulation, AMVF reference	1.72
Run 2 data, AMVF reference	1.82
Run 3 data, AMVF reference	1.96

In data the reference is AMVF; simulation with the same reference reproduces the data.

Backup: Athena Integration



`InDetNNPriVxFinderTool` and `InDetGNNTrackVertexAssociationTool`, with the same `IVertexFinder` interface as AMVF. ONNX Runtime on CPU or CUDA, set by configuration. Athena output agrees with the Python implementation to floating-point precision.

Note Sec. 5.1 (18 Sep 2026).